

```
/*
```

```
ESP32-WROOM Turret Receiver - Face Tracking Servo Controller  
Arduino core: ESP32 by Espressif Systems 3.x
```

- Joins hotspot for channel match
- Receives TrackPacket via ESP-NOW
- Drives 2 SG90 servos (pan + tilt)
- MIRRORED PAN: facing each other (move left -> turret moves right)

```
*/
```

```
#include <WiFi.h>
```

```
#include <esp_now.h>
```

```
// ===== Hotspot credentials (same as ESP32-CAM) =====
```

```
const char* ssid    = "YOUR_HOTSPOT_NAME";
```

```
const char* password = "YOUR_HOTSPOT_PASSWORD";
```

```
// ===== Servo pins =====
```

```
const int PAN_PIN = 25;
```

```
const int TILT_PIN = 26;
```

```
// ===== PWM settings =====
```

```
const int PWM_FREQ = 50;
```

```
const int PWM_RES = 16;
```

```
// ===== Control tuning =====
```

```
float pan = 90.0f;
```

```
float tilt = 90.0f;
```

```
float Kp = 0.06f; // increase if sluggish, decrease if twitchy
```

```
int deadband = 6; // pixels
```

```
float smooth = 0.35f; // 0..1 (higher = snappier)
```

```
bool invertTilt = false; // set true if tilt moves wrong direction
```

```
// ===== Packet =====
```

```
typedef struct __attribute__((packed)) {
```

```
    int16_t ex;
```

```
    int16_t ey;
```

```
    uint8_t found;
```

```
} TrackPacket;
```

```
volatile TrackPacket lastPkt;
```

```
volatile bool gotPkt = false;
```

```
// ESP32 core 3.x callback signature
void onReceive(const esp_now_recv_info_t* info, const uint8_t* data, int len) {
    (void)info;
    if (len == (int)sizeof(TrackPacket)) {
        memcpy((void*)&lastPkt, data, sizeof(TrackPacket));
        gotPkt = true;
    }
}
```

```
// ===== Servo helpers (core 3.x) =====
int angleToUs(int angle) {
    angle = constrain(angle, 0, 180);
    return 500 + (int)((2400 - 500) * (angle / 180.0));
}
uint32_t usToDuty(int us) {
    const int periodUs = 20000;
    us = constrain(us, 400, 2600);
    uint32_t maxDuty = (1UL << PWM_RES) - 1;
    return (uint32_t)((us / (double)periodUs) * maxDuty);
}
void servoAttach(int pin) {
    if (!ledcAttach(pin, PWM_FREQ, PWM_RES)) {
        Serial.printf("LEDC attach failed on pin %d\n", pin);
        while (true) delay(1000);
    }
}
void servoWriteAngle(int pin, int angle) {
    ledcWrite(pin, usToDuty(angleToUs(angle)));
}
```

```
void setup() {
    Serial.begin(115200);
    delay(200);
```

```
// Wi-Fi STA + hotspot join (for channel match reliability)
WiFi.mode(WIFI_STA);
WiFi.setSleep(false);
```

```
Serial.print("WROOM MAC: ");
Serial.println(WiFi.macAddress());
```

```
Serial.print("Connecting to hotspot: ");
Serial.println(ssid);
```

```

WiFi.begin(ssid, password);
unsigned long t0 = millis();
while (WiFi.status() != WL_CONNECTED && millis() - t0 < 15000) {
    delay(300);
    Serial.print(".");
}
Serial.println();

if (WiFi.status() == WL_CONNECTED) {
    Serial.print("Hotspot connected. IP: ");
    Serial.println(WiFi.localIP());
} else {
    Serial.println("Hotspot NOT connected (ESP-NOW may be unreliable if channels differ).");
}

// Servo init
servoAttach(PAN_PIN);
servoAttach(TILT_PIN);
servoWriteAngle(PAN_PIN, (int)pan);
servoWriteAngle(TILT_PIN, (int)tilt);

// ESP-NOW init
if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init failed");
    while (true) delay(1000);
}
if (esp_now_register_rcv_cb(onReceive) != ESP_OK) {
    Serial.println("ESP-NOW register rcv cb failed");
    while (true) delay(1000);
}

Serial.println("Turret receiver ready.");
}

void loop() {
    if (!gotPkt) return;
    gotPkt = false;

    int ex = lastPkt.ex;
    int ey = lastPkt.ey;
    int found = lastPkt.found;

    // Optional: if no face found, hold position

```

```

if (!found) return;

if (abs(ex) < deadband) ex = 0;
if (abs(ey) < deadband) ey = 0;

// MIRRORED PAN (facing each other)
float targetPan = constrain(pan - Kp * ex, 0.0f, 180.0f);

// Tilt sign toggle
float targetTilt = invertTilt
  ? constrain(tilt + Kp * ey, 0.0f, 180.0f)
  : constrain(tilt - Kp * ey, 0.0f, 180.0f);

// Smooth motion
pan = pan + smooth * (targetPan - pan);
tilt = tilt + smooth * (targetTilt - tilt);

servoWriteAngle(PAN_PIN, (int)pan);
servoWriteAngle(TILT_PIN, (int)tilt);
}

```